Alani: A Bare-Metal Intelligence Operating System

A Cognitive Kernel for Orchestrating Machine Reasoning at the Systems Level

Marlon Hanks

Independent Researcher

April 2026

Open Specification — This document describes an open systems architecture. It is released as a public specification for research, implementation, and extension.

Abstract

Alani is a bare-metal intelligence operating system that elevates reasoning, memory, and decision-making to first-class kernel-managed resources. Unlike conventional operating systems that abstract hardware primitives such as CPU, memory, and I/O, Alani introduces a cognitive kernel that governs the lifecycle of intent, context, inference, and action through a syscall-mediated protocol. Implemented in Rust for memory safety and deterministic control, Alani separates intelligence orchestration from model execution, treating models as kernel-managed cognitive devices. This architecture enables secure, observable, and composable intelligence systems that operate with the rigor of operating system design rather than application-layer orchestration.

Keywords: operating systems, cognitive kernel, machine intelligence, bare-metal, Rust, syscall interface, cognitive devices

---

1. Introduction

Modern computing systems are optimized for deterministic execution over static data structures. Intelligence—reasoning over dynamic context—is layered on top via applications, services, or APIs. This separation introduces systemic limitations:

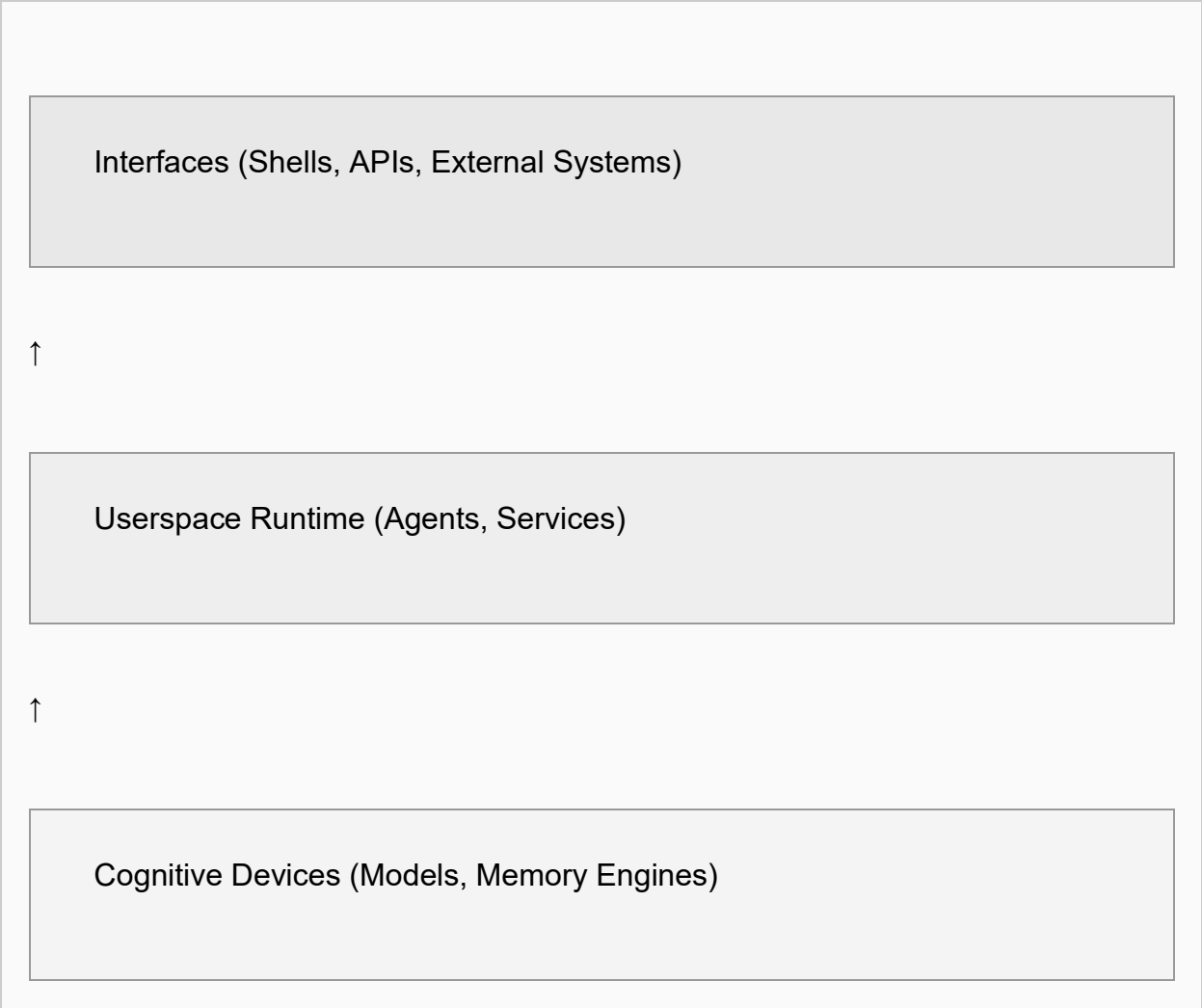
- Context fragmentation across processes and systems

- Lack of persistent, system-level reasoning
- No formal scheduling or isolation of cognitive workloads
- Absence of verifiable execution for decision-making processes

Alani addresses these limitations by redefining the operating system abstraction: intelligence is treated as a kernel-managed resource, not an application concern. This shifts the OS from a passive execution environment to an active cognitive control plane.

2. System Model

Alani introduces a vertically integrated system model in which each layer of the stack assumes explicit responsibility for a class of operations. The architecture is organized as follows:



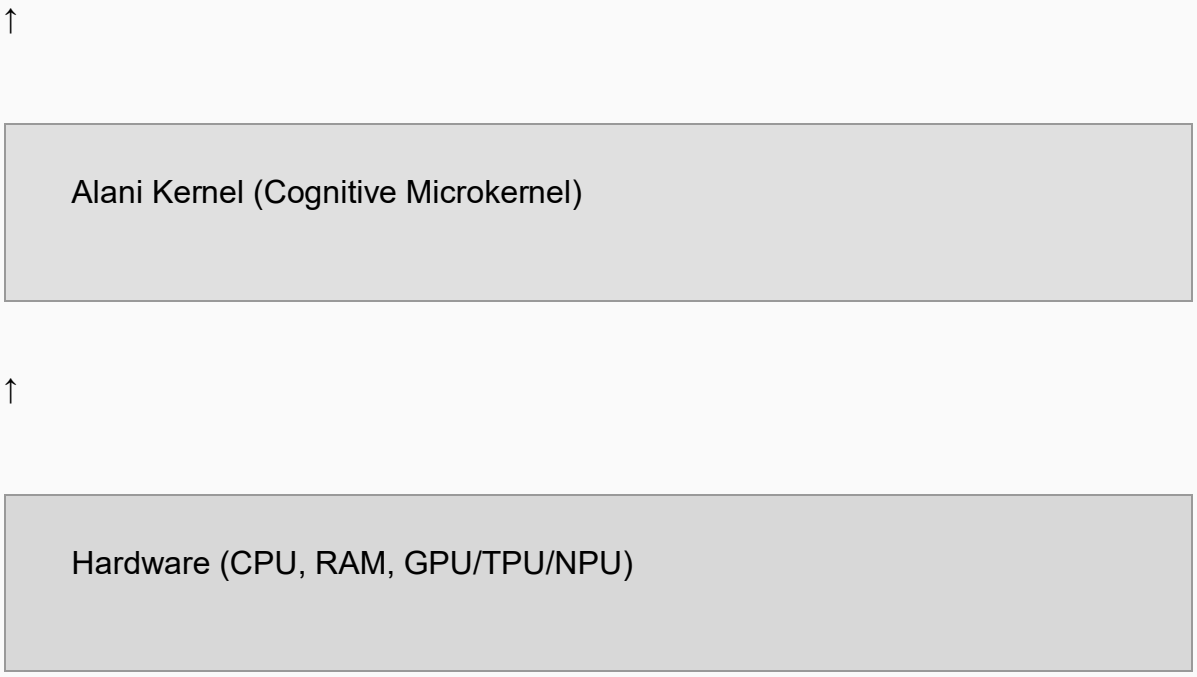


Figure 1: Alani system architecture stack.

Table 1: Layer Responsibilities

Layer	Responsibility
Hardware	CPU, RAM, accelerators (GPU/TPU/NPU)
Kernel	Scheduling, memory, IPC, cognitive control
Cognitive Devices	Model inference, semantic memory
Userspace	Agents, workflows, applications
Interfaces	External interaction (shells, APIs, external systems)

3. Design Principles

The design of Alani is governed by five foundational principles:

1. **Intelligence as a Primitive.** Reasoning and context are managed like memory and processes. The kernel is aware of cognitive workloads and manages them as first-class entities with defined lifecycles, permissions, and scheduling constraints.
2. **Kernel/User Separation.** Cognitive control resides in kernel space; execution occurs in userspace. This mirrors the classical separation of privilege in operating system design and prevents user-level processes from bypassing policy enforcement.
3. **Model Abstraction.** Models are devices, not logic embedded in the kernel. The kernel does not perform inference directly; it orchestrates and governs inference through a device interface, enabling model-agnostic scheduling and resource management.
4. **Deterministic Governance, Probabilistic Execution.** Kernel behavior is deterministic; inference remains probabilistic. The control path—scheduling, routing, policy enforcement—is fully deterministic and auditable, while the data path through model devices preserves the inherent stochasticity of neural inference.
5. **Memory Safety and Concurrency.** Rust-based implementation ensures safe low-level execution. By leveraging Rust's ownership model and borrow checker, Alani eliminates entire classes of memory safety vulnerabilities at compile time without sacrificing bare-metal performance.

4. Kernel Architecture

Alani follows a microkernel-inspired design. The kernel maintains a minimal trusted computing base and delegates domain-specific functionality to userspace servers and cognitive devices.

4.1 Kernel Responsibilities

The kernel is responsible for the following core operations:

- Process and thread scheduling
- Memory management
- Inter-process communication (IPC)

- Cognitive syscall handling
- Policy enforcement
- Execution tracing

4.2 Cognitive Subsystem

The kernel introduces a dedicated cognitive subsystem responsible for all intelligence-related operations. The subsystem is organized as follows:

```
cognition/   intent.rs        // Intent lifecycle management   context.rs
// Context assembly and resolution   planner.rs       // Task decomposition
and planning   router.rs        // Model and device routing   policy.rs
// Cognitive policy enforcement
```

This subsystem governs the full lifecycle of cognitive operations—from intent creation through context resolution, task planning, device routing, and policy enforcement. Each module operates under kernel privilege and enforces invariants before dispatching work to userspace devices.

5. Cognitive Syscall Interface

All intelligence operations in Alani are mediated through a formal syscall interface. No cognitive operation may execute without traversing the kernel boundary, ensuring that every invocation is permission-checked, scheduled, and traceable.

5.1 Core Syscalls

```
int sys_intent_create(intent_t* intent); int sys_context_resolve(intent_id_t
id, context_t* ctx); int sys_model_infer(task_t* task, result_t* out); int
sys_memory_query(query_t* q, result_t* out); int sys_memory_write(memory_t*
mem); int sys_action_execute(action_t* action); int sys_trace_emit(trace_t*
trace);
```

5.2 Syscall Properties

Each cognitive syscall exhibits the following properties:

- **Permission-checked at kernel boundary.** Every syscall invocation is validated against the calling process's capability set before dispatch.
- **Auditable via trace logs.** The kernel emits structured trace records for every syscall, enabling full reconstruction of reasoning paths.

- **Schedulable as kernel-managed tasks.** Syscalls are enqueued and dispatched by the cognitive scheduler, subject to priority, latency, and resource constraints.

6. Cognitive Devices

Models and memory systems are abstracted as kernel-managed devices. This design mirrors the device model of conventional operating systems, where hardware peripherals are accessed through a standardized interface rather than direct manipulation.

6.1 Model Devices

Model devices are analogous to GPU devices in Linux. They expose inference capabilities through a device file interface:

```
/dev/model/llm0        // Large language model, instance 0 /dev/model/vision0
// Vision model, instance 0
```

Model device responsibilities include:

- Inference execution against submitted tasks
- Capability reporting (supported modalities, context window, latency characteristics)
- Queue handling and backpressure signaling

6.2 Memory Devices

Memory devices provide persistent semantic storage and retrieval:

```
/dev/memory/vector0    // Vector embedding store, instance 0
/dev/memory/graph0     // Knowledge graph store, instance 0
```

Memory devices provide:

- Semantic retrieval over high-dimensional embedding spaces
- Structured queries over graph-based knowledge representations
- Persistence across sessions and system restarts

7. Cognitive Scheduling

The Alani kernel scheduler extends beyond traditional CPU time-sharing to manage the full spectrum of cognitive resource allocation.

7.1 Scheduling Dimensions

The scheduler operates across multiple resource dimensions simultaneously:

- CPU time
- Memory bandwidth
- Model inference slots
- Context size (token budget allocation)
- Latency constraints (deadline-aware scheduling)

7.2 Policy Goals

Scheduling decisions are governed by configurable policy objectives:

- Minimize end-to-end latency for interactive workloads
- Optimize cost in distributed inference environments
- Prioritize high-importance intents based on declared urgency and system policy

8. Memory Architecture

Memory in Alani is multi-tiered and kernel-governed. Each tier serves a distinct function and is subject to its own access control and persistence policies.

8.1 Memory Types

- **Kernel Cognitive Memory.** Secure, policy-critical data accessible only to kernel-level cognitive operations. Stores active intent state, scheduling metadata, and policy configurations.
- **User Cognitive Memory.** Agent-accessible state scoped to individual processes or sessions. Provides working memory for userspace agents and services.

- **Shared Semantic Memory.** Embeddings, documents, and knowledge structures accessible across processes subject to capability-based permissions.
- **Execution Trace Memory.** Full reasoning logs capturing the complete decision path from intent creation through action execution.

8.2 Memory Properties

- **Permission-enforced.** All memory access is mediated by capability tokens validated at the syscall boundary.
- **Queryable via syscalls.** Memory contents are accessed exclusively through `sys_memory_query` and `sys_memory_write`.
- **Persisted across sessions.** Memory state survives process termination and system restarts, subject to configured retention policies.

9. Execution Model

Alani enforces a strict separation between kernel-space control and userspace execution.

9.1 Kernel Space

Kernel space is responsible for syscall handling, scheduling, policy enforcement, and trace emission. All operations in kernel space are deterministic and execute with elevated privilege.

9.2 Userspace

Userspace hosts agents, tools, workflows, and model device servers. All inference execution occurs in userspace, isolated from kernel internals by the syscall boundary.

9.3 Model Execution Flow

A complete cognitive execution cycle proceeds through the following stages:

```
Userspace Agent

→

syscall(

sys_intent_create

)

→

Kernel Planning

→

Model Device Invocation

→

Result Returned to Agent
```

At each stage, the kernel validates permissions, applies scheduling policy, records trace data, and enforces resource limits before dispatching to the next stage.

10. Security Model

Security in Alani is enforced at the kernel level. No cognitive operation—intent creation, context resolution, model inference, or memory access—may bypass the kernel's security enforcement.

10.1 Mechanisms

- **Capability-based access control.** Processes hold unforgeable capability tokens that grant specific permissions on specific resources.
- **Memory isolation.** Cognitive memory tiers are isolated by kernel-enforced boundaries. Cross-tier access requires explicit capability grants.

- **Agent sandboxing.** Userspace agents execute within sandboxed environments with restricted syscall access determined by their capability set.
- **Device-level permissions.** Access to model and memory devices is governed by per-device capability requirements.

10.2 Enforcement Points

Security policy is enforced at three critical points in the system:

- **Syscall boundary.** Every syscall is validated against the caller's capability set before dispatch.
- **Memory access.** Read and write operations on cognitive memory are checked against tier-specific permissions.
- **Device invocation.** Model and memory device operations require device-specific capabilities.

11. Observability

Every cognitive operation in Alani is traceable. The kernel maintains a structured trace log that records the full lifecycle of every intent, from creation through execution and result delivery.

11.1 Trace Components

Each trace record captures the following components:

- Intent lifecycle (creation, decomposition, completion, failure)
- Context assembly (sources consulted, relevance scoring, token allocation)
- Model usage (device selected, latency, token consumption)
- Execution results (output, confidence, error conditions)

11.2 Example Trace

The following is a representative trace record emitted by the kernel:

```
{   "intent_id": "123",   "tasks": [     {       "task_id": "t-001",
"type": "inference",       "device": "/dev/model/llm0"     }   ],
"models_used": ["llm0"],   "latency_ms": 842,   "result": "success" }
```


12. Minimal Viable Kernel (MVK)

The Minimal Viable Kernel defines the smallest functional subset of Alani sufficient to demonstrate end-to-end cognitive execution.

12.1 Scope

The MVK includes the following components:

- Bootable Rust kernel with hardware initialization
- Basic round-robin scheduler
- Cognitive syscall layer (subset: intent, context, infer, memory)
- Single model device hosted in userspace
- Simple key-value memory device

12.2 Execution Path

The MVK demonstrates the following minimal execution path:

```
input   → sys_intent_create   → sys_context_resolve   → sys_model_infer   →
sys_memory_write   → output
```

This path exercises every layer of the Alani architecture: syscall dispatch, kernel planning, device invocation, and memory persistence.

13. Differentiation

Alani occupies a distinct position in the systems landscape by embedding intelligence orchestration directly within the kernel. The following table summarizes the key architectural difference:

Table 2: Intelligence Location by System

System	Intelligence Location
Linux	None (applications only)
AI Platforms (e.g., LangChain, AutoGen)	Application layer
Alani	Kernel layer

By situating intelligence governance at the kernel layer, Alani provides guarantees—isolation, scheduling, traceability, and policy enforcement—that are structurally impossible to achieve in application-layer orchestration frameworks.

14. Future Work

Several directions for future research and development extend naturally from the current specification:

- **Distributed cognitive kernels.** Extending the Alani kernel model to operate across networked nodes, enabling distributed intent routing and federated cognitive memory.
- **Hardware acceleration for inference.** Defining kernel-level interfaces for direct accelerator management (NPU, custom ASIC) without userspace mediation.
- **Formal verification of reasoning traces.** Applying formal methods to verify properties of cognitive execution traces, such as completeness, consistency, and policy compliance.
- **Standardized cognitive syscall ABI.** Defining a stable application binary interface for cognitive syscalls, enabling cross-implementation compatibility and binary portability of cognitive agents.

15. Conclusion

Alani redefines the operating system by embedding intelligence directly into the kernel. Through a syscall-mediated protocol, it governs how reasoning is invoked, executed, and audited. By separating cognitive control from model execution and treating models as devices, Alani establishes a foundation for secure, observable, and scalable machine intelligence systems.

The architecture presented in this specification is intentionally minimal. It does not prescribe model architectures, inference strategies, or agent frameworks. Instead, it provides the systems-level substrate—scheduling, memory management, security, and observability—upon which such higher-order constructs can be reliably and safely composed.

"Intelligence is no longer an application feature—it is an operating system primitive."

References

[1] A. S. Tanenbaum, *Modern Operating Systems*, 4th ed. Pearson, 2014.

[2] J. Liedtke, "On micro-kernel construction," in *Proc. 15th ACM Symposium on Operating Systems Principles (SOSP)*, 1995, pp. 237–250.

[3] R. Klabnik and S. Nichols, *The Rust Programming Language*. No Starch Press, 2019.

[4] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[5] J. S. Chase et al., "Sharing and protection in a single-address-space operating system," *ACM Transactions on Computer Systems*, vol. 12, no. 4, pp. 271–307, 1994.

[6] D. R. Engler, M. F. Kaashoek, and J. O'Toole Jr., "Exokernel: An operating system architecture for application-level resource management," in *Proc. 15th ACM SOSP*, 1995, pp. 251–266.

[7] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in NeurIPS*, 2020.

[8] Y. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.